PROJECT DOCUMENTATION

Voice-Based Concept Understanding Analyser (VBCUA)

An AI-Powered System for Evaluating Spoken Conceptual Explanations Using Speech-to-Text Transcription, Semantic Similarity, and Audio Feature Analysis

Repository: github.com/Sarika-Reddy-S/Voice-Based-Concept-Understanding-Analyser

Document prepared: July 2026

S.No	Name	Email	Role
1	Sathi Sarika Reddy	sarikareddychinta@gmail.com	Team Lead
2	Kanigolla N V B J Vigneswari	23mh1a4292@acoe.edu.in	Member
3	Kanukolanu Pujitha	pujithak19@gmail.com	Member
4	Tirumalaraju Neha	nehatirumalaraju@gmail.com	Member
5	Jyothi Anuhya Ulli	anuhyaulli199@gmail.com	Member

Table of Contents

1. Project Overview

The Voice-Based Concept Understanding Analyser (VBCUA) is an AI-powered web application that evaluates how well a learner understands a given concept, based on a spoken explanation. Rather than relying on written exams or multiple-choice questions, VBCUA allows a student to upload an audio recording of themselves explaining a concept. The system automatically transcribes the speech, compares its meaning against a reference explanation, analyses delivery quality from the raw audio, and produces a composite Concept Understanding Score together with actionable feedback and a downloadable PDF report.

The application is intended to support viva preparation, interview practice, self-study, and classroom assessment. It combines four core AI/ML capabilities in a single pipeline: automatic speech recognition, semantic textual similarity, acoustic feature analysis, and automated report generation.

1.1 Repository Information

Field	Detail
Repository	Sarika-Reddy-S/Voice-Based-Concept-Understanding-Analyser
Platform	GitHub
Primary language	Python (100%)
License	MIT License
Default branch	master
Entry points	app.py, main.py (streamlit run app.py or main.py)

2. Problem Statement and Objectives

2.1 Problem Statement

There is no readily available, open-source, automated system that evaluates the quality of a spoken conceptual explanation in a holistic way, combining semantic accuracy, fluency, and communication quality. Existing speech assessment tools tend to focus on pronunciation or general language proficiency rather than domain-specific concept understanding.

2.2 Objectives

- Develop an end-to-end pipeline that accepts audio input, transcribes it using OpenAI Whisper, and produces a quantitative evaluation of conceptual understanding.
- Implement semantic similarity scoring using Sentence-BERT to compare a transcribed explanation against a reference concept definition.
- Extract meaningful audio features, including speaking rate, pause ratio, pitch stability, and RMS energy, using Librosa, and incorporate them into the scoring formula.
- Detect and quantify filler word usage as a proxy for speech fluency.

- Build a responsive, dark-themed web interface using Streamlit that presents results clearly and allows report download.
- Persist all session and evaluation data in a normalised SQLite relational database that follows a defined entity-relationship schema.
- Generate automated, downloadable PDF reports for each evaluation session.

2.3 Scope

The scope of VBCUA is limited to evaluation of spoken explanations in English, delivered as audio files in common formats (WAV, MP3, M4A, OGG, FLAC). The current version does not perform real-time microphone recording. It is intended as a self-assessment and learning tool, not as a replacement for certified language or academic assessment systems.

3. Literature Survey

Automatic Speech Recognition

Whisper (Radford et al., 2022) is a large-scale ASR model trained on 680,000 hours of multilingual, multitask supervised data. Its robustness to accents, noise, and inconsistent recording conditions makes it well suited to student-recorded audio.

Semantic Textual Similarity

Sentence-BERT (Reimers and Gurevych, 2019) uses siamese network fine-tuning of BERT to produce sentence embeddings that can be compared efficiently using cosine similarity. The all-MiniLM-L6-v2 variant balances inference speed and semantic quality for interactive use.

Audio Feature Analysis

Established acoustic features such as pitch, energy, and zero-crossing rate are indicators of speech quality. Librosa packages these as accessible Python functions, and prior work has shown pause duration and speaking rate correlate with perceived clarity and confidence.

Disfluency Detection

Filler word usage is well studied in speech fluency research. Lexical pattern matching provides a simple yet effective signal for fluency scoring and is used in platforms such as language learning tools for pronunciation and fluency assessment.

Automated Assessment Systems

Latent Semantic Analysis for essay grading laid early groundwork for automated conceptual assessment; modern transformer-based approaches have since improved accuracy substantially. Spoken concept evaluation, however, remains comparatively unexplored relative to written assessment.

While individual components such as ASR, semantic textual similarity, and audio feature extraction are each well studied, no open-source tool integrates all three into a unified spoken concept evaluation system with a web interface and downloadable reports. VBCUA addresses this gap.

4. System Architecture

VBCUA follows a sequential, modular pipeline architecture. Each processing step is implemented as a separate Python module with a clearly defined interface, allowing independent development, testing, and replacement of individual components.

4.1 Pipeline Flow

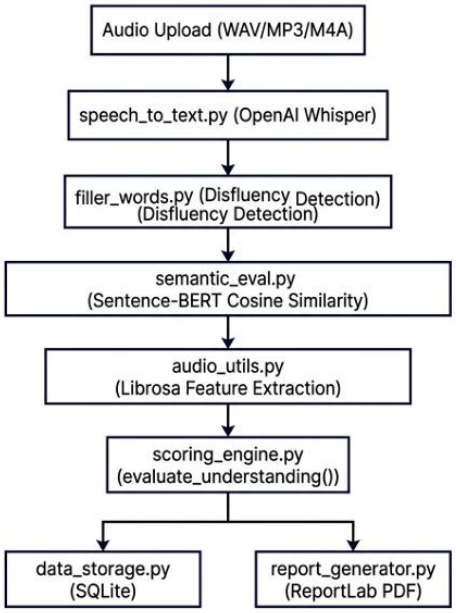


Figure 4.1 - Sequential pipeline architecture of VBCUA.

4.2 Module Responsibilities

Module	Responsibility
speech_to_text.py	Wraps Whisper; normalises audio to 16 kHz mono; returns transcript, word count, language.
filler_words.py	Regex-based disfluency detection; returns filler count, ratio, per-word breakdown.
semantic_eval.py	Sentence-BERT embeddings and cosine similarity; keyword coverage analysis.
audio_utils.py	Librosa-based extraction of pause ratio, RMS energy, pitch, ZCR, tempo, duration.
scoring_engine.py	Combines all signals into a final score (0-100) and an understanding level.
data_storage.py	SQLite persistence layer; CRUD operations for every entity in the ER schema.
report_generator.py	Builds a multi-page PDF report using ReportLab.
app.py / main.py	Streamlit front end; orchestrates the pipeline and renders results.

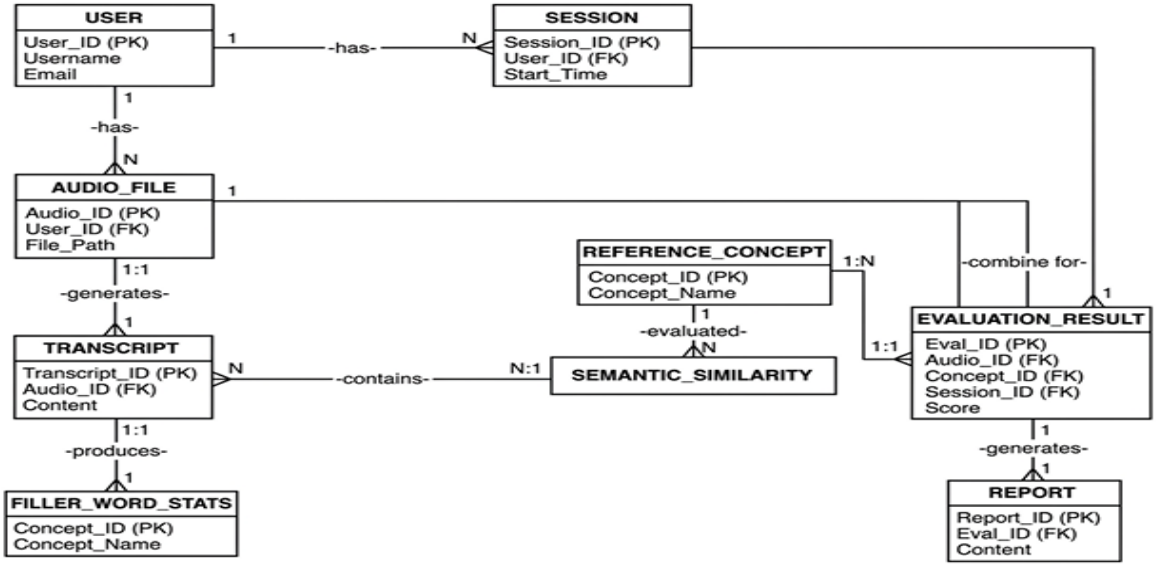
Legacy shim modules (audio_features.py, transcription.py, semantic_analysis.py, scoring.py) delegate to the canonical implementations above, preserved for backward compatibility.

5. Data Model (Entity-Relationship Design)

The persistence layer (modules/data_storage.py) implements a fully normalised relational schema in SQLite. Foreign key relationships enforce referential integrity and support the full session lifecycle, from audio upload through to report generation.

Entity	Primary Key	Foreign Keys	Purpose
USER	user_id	-	Stores learner name, email, role, created_at.
SESSION	session_id	user_id	Groups evaluations into a single analysis session.
AUDIO_FILE	audio_id	user_id	Stores file name, path, duration, upload status.
REFERENCE_CONCEPT	ref_concept_id	-	Stores concept title and reference explanation text.
TRANSCRIPT	transcript_id	audio_id	Stores the Whisper-generated transcript text.
AUDIO_FEATURE	feature_id	audio_id	Stores pause_ratio, rms_energy, ZCR, duration.
FILLER_WORD_STATS	filler_id	transcript_id	Filler count, total words, filler ratio.
SEMANTIC_SIMILARITY	similarity_id	transcript_id, ref_concept_id	Cosine similarity score.
EVALUATION_RESULT	result_id	audio_id, ref_concept_id, session_id	Overall score, level, notes.
REPORT	report_id	result_id	PDF path, generated_at, file_size_kb.

5.1 Relationship Summary



6. Technology Stack

Layer	Technology / Library	Purpose
Language	Python 3.10+	Core programming language for all modules.
Frontend UI	Streamlit	Interactive web application and results dashboard.
Speech Recognition	OpenAI Whisper	Converts student audio into transcript text.
NLP / Semantics	Sentence-Transformers (all-MiniLM-L6-v2)	Sentence-BERT embeddings and cosine similarity.
Deep Learning	PyTorch	Backend inference engine for Whisper and Sentence-BERT.
Audio Analysis	Librosa	Pitch, energy, tempo, zero-crossing rate, pause ratio extraction.
Audio I/O	SoundFile, audioread	Audio file loading across multiple formats.
Numerical Computing	NumPy, SciPy	Array operations and signal processing.
NLP Utilities	NLTK	Tokenisation and text analysis helpers.
Visualisation	Matplotlib	Waveform rendering.
Report Generation	ReportLab	PDF report creation.
Database	SQLite (stdlib)	Relational session and result persistence.
Testing	Pytest	Automated unit and integration testing.

One notable setup requirement is ffmpeg, which Whisper depends on for audio format conversion. On Windows this requires downloading the binary and adding it to the system PATH; on Linux and macOS it can be installed with a single package manager command.

7. Installation

The following steps set up the project locally.

```
# 1. Clone the repository
git clone https://github.com/Sarika-Reddy-S/Voice-Based-Concept-Understanding-Analyser.git
cd Voice-Based-Concept-Understanding-Analyser


# 2. Create a virtual environment (recommended)
python -m venv venv
# Windows:
```

```
venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate


# 3. Install dependencies
pip install -r requirements.txt
```

7.1 ffmpeg Requirement

- macOS: brew install ffmpeg
- Ubuntu / Debian: sudo apt install ffmpeg
- Windows: download from ffmpeg.org and add the binary to the system PATH

8. Usage

```
streamlit run app.py
# or equivalently:
streamlit run main.py
```

Once the application is running in the browser:

- 1. Enter the learner's name in the sidebar.
- 2. Upload an audio recording (WAV, MP3, M4A, OGG, or FLAC).
- 3. Paste the reference concept explanation into the right-hand panel.
- 4. Click "Analyze Concept Understanding".
- 5. Review the transcript, understanding score (Strong / Moderate / Poor), semantic similarity, filler word ratio, energy confidence, and waveform.
- 6. Download the generated PDF report.

9. Project Structure

```
Voice-Based-Concept-Understanding-Analyser/
|-- app.py                Streamlit front end and main application logic
|-- main.py               Entry-point alias (streamlit run main.py)
|-- requirements.txt
|-- README.md
|-- LICENSE
|-- .gitignore
|
|-- modules/
```

```
|   |-- __init__.py           Public API exports
|   |-- audio_utils.py        Audio loading and feature extraction utilities
|   |-- speech_to_text.py     Whisper-based transcription logic
|   |-- semantic_eval.py      Semantic similarity using Sentence-BERT
|   |-- scoring_engine.py     Understanding score calculation and classification
|   |-- filler_words.py       Filler word / disfluency detection
|   |-- report_generator.py   PDF report generation using ReportLab
|   |-- data_storage.py       SQLite persistence layer (full ER schema)
|   |
|   |-- audio_features.py     (shim) delegates to audio_utils.py
|   |-- transcription.py      (shim) delegates to speech_to_text.py
|   |-- semantic_analysis.py  (shim) delegates to semantic_eval.py
|   |-- scoring.py            (shim) delegates to scoring_engine.py
|
|-- tests/
|   |-- test_audio_utils.py       AudioFeatures, speaking rate
|   |-- test_speech_to_text.py    filler_word_ratio, TranscriptionResult
|   |-- test_semantic_eval.py     sentence splitting, keyword extraction
|   |-- test_scoring_engine.py    evaluate_understanding, compute_score
|   |-- test_scoring.py           legacy scoring tests
|   |-- test_semantic.py          legacy semantic tests
|   |-- test_filler_words.py      filler word analysis
|   |-- test_data_storage.py      SQLite CRUD operations
|
|-- data/                   SQLite database (auto-created at runtime)
|-- assets/                 waveform images (auto-generated)
|-- reports/                generated PDF reports
```

10. Core Logic

10.1 Scoring Formula

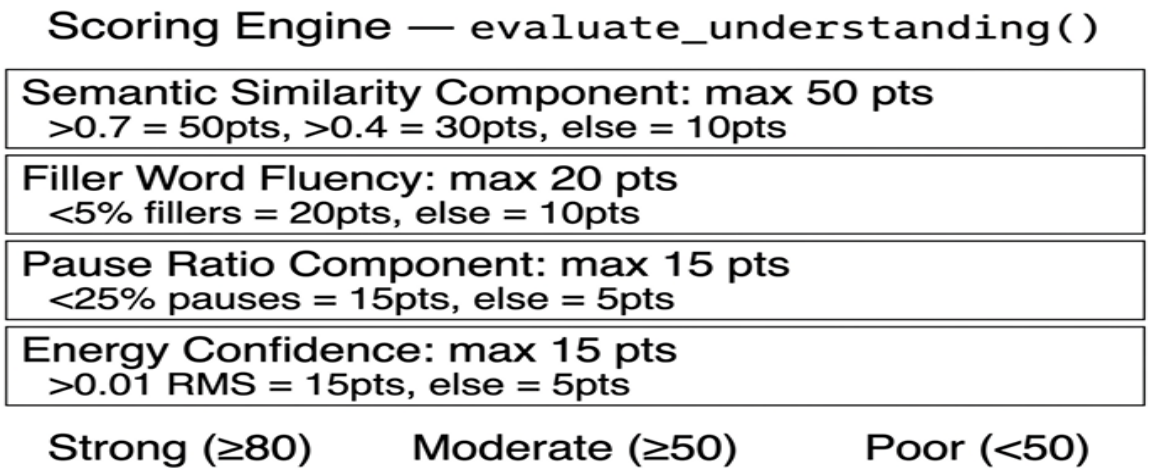
The scoring engine implements a primary scoring function, evaluate_understanding(), which combines semantic similarity, filler ratio, pause ratio, and RMS energy into a single 0-100 score:

```
score += 50 if similarity  > 0.7  else 30 if similarity  > 0.4  else 10
score += 20 if filler_ratio < 0.05  else 10
```

```
score += 15 if pause_ratio  < 0.25  else 5
score += 15 if rms_energy   > 0.01  else 5


Strong Understanding:    score >= 80
Moderate Understanding:  score >= 50
Poor Understanding:      score <  50
```

A complementary function, compute_score(), produces a detailed sub-score breakdown using weighted contributions of Understanding (55 percent), Fluency (25 percent), and Clarity (20 percent), which drives the score card in the interface and the PDF report.



10.2 Audio Features Extracted

- pause_ratio - proportion of near-silent frames (RMS below 2 percent of maximum), used to detect hesitation.
- rms_energy - mean root-mean-square energy, used as a proxy for vocal confidence.
- rms_energy_std - standard deviation of frame-level RMS, used to assess energy consistency.
- pitch_mean_hz and pitch_std_hz - mean and standard deviation of estimated fundamental frequency.
- zero_crossing_rate - rate of sign changes in the waveform, related to voice quality.
- tempo_bpm - estimated beats per minute derived from onset detection.
- duration_sec - total audio length in seconds.
- estimated_pause_count - number of distinct silence segments.

10.3 Filler Word Detection

The filler_words.py module uses regular expressions to detect common English disfluencies, including um, uh, ah, er, like, you know, I mean, basically, literally, right, so, well, and okay. It returns a FillerStats structure containing the filler word count, total word count, filler ratio, and a per-word breakdown. The filler ratio feeds into both scoring functions as a fluency penalty.

11. Testing

Testing is carried out with Pytest:

```
pytest tests/ -v
```

Test File	Coverage Focus
test_scoring_engine.py	evaluate_understanding formula, compute_score weights, grade labels, feedback generation.
test_audio_utils.py	AudioFeatures dataclass, speaking-rate edge cases (zero duration, normal WPM).
test_speech_to_text.py	filler_word_ratio computation, TranscriptionResult fields.
test_semantic_eval.py	Sentence splitting logic, keyword extraction, SemanticResult fields.
test_filler_words.py	Filler word detection accuracy, FillerStats fields, ratio computation.
test_data_storage.py	SQLite CRUD operations, session lifecycle (start to end), foreign key relationships.
test_scoring.py	Legacy scoring shim - delegates to scoring_engine.
test_semantic.py	Legacy semantic shim - delegates to semantic_eval.

Whisper and Sentence-BERT calls are mocked in tests to keep the suite fast and independent of network access, since both models download several hundred megabytes on first use. Only the logic layers, such as the scoring formula, feature extraction arithmetic, and database CRUD operations, are exercised directly.

Performance optimisation is achieved through @st.cache_resource for model loading and functools.lru_cache for evaluator instances, reducing reload overhead during interactive use and testing.

12. Project Workflow

Development was organised using an agile-inspired epic and story structure, dividing the work into manageable units with clear milestones.

Epic 1: Environment Setup

- Story 1: Python virtual environment creation and dependency installation via pip install -r requirements.txt.
- Story 2: Project directory structure initialisation - app.py, modules/, tests/, data/, reports/, assets/.
- Story 3: Streamlit application skeleton; confirmed the interface loads at localhost:8501.

Epic 2: Core Engine Development

- Story 1: Speech-to-Text module (modules/speech_to_text.py); integrated Whisper, handled WAV normalisation and multi-format inputs.
- Story 2: Semantic Similarity Engine (modules/semantic_eval.py); Sentence-BERT embeddings, cosine similarity, keyword coverage analysis.

- Story 3: Audio Feature Extraction and Scoring Engine (modules/audio_utils.py, modules/scoring_engine.py); Librosa features and the evaluate_understanding formula.
- Story 4: Filler Word Detection (modules/filler_words.py); pattern-based detection with a configurable word list.
- Story 5: Data Persistence (modules/data_storage.py); full SQLite schema matching the ER diagram, CRUD operations for all entities.
- Story 6: Report Generation (modules/report_generator.py); ReportLab PDF report with transcript, scores, feedback, and keyword coverage.

Epic 3: UI Development

- Story 1: Dark-themed Streamlit layout with waveform display, score card, and metrics row.
- Story 2: Input handling; audio file uploader, concept title input, reference text area, Whisper model size selector.
- Story 3: Output rendering; analysis banner, score display, transcript box, filler word table, keyword coverage, feedback items.
- Story 4: Session history sidebar showing the last ten sessions with score and level.

Epic 4: Testing and Deployment

- Story 1: Functional testing and validation using pytest tests/ -v across eight test files.
- Story 2: Performance optimisation using @st.cache_resource and lru_cache.
- Story 3: Deployment preparation; confirmed compatibility with Streamlit Community Cloud and documented a Docker-ready structure.

13. Results and Discussion

The application was tested end-to-end with a variety of student audio samples covering different concept explanations. Results consistently demonstrated the system's ability to distinguish between strong and poor conceptual understanding, and to provide specific, actionable feedback.

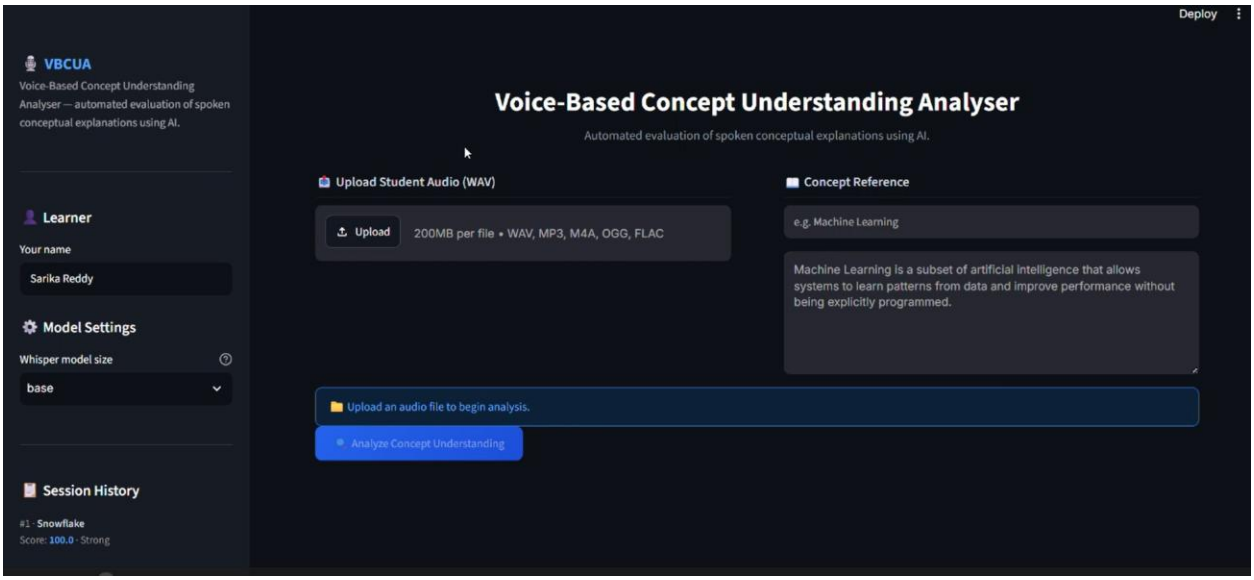


Fig 13.1 Home Page

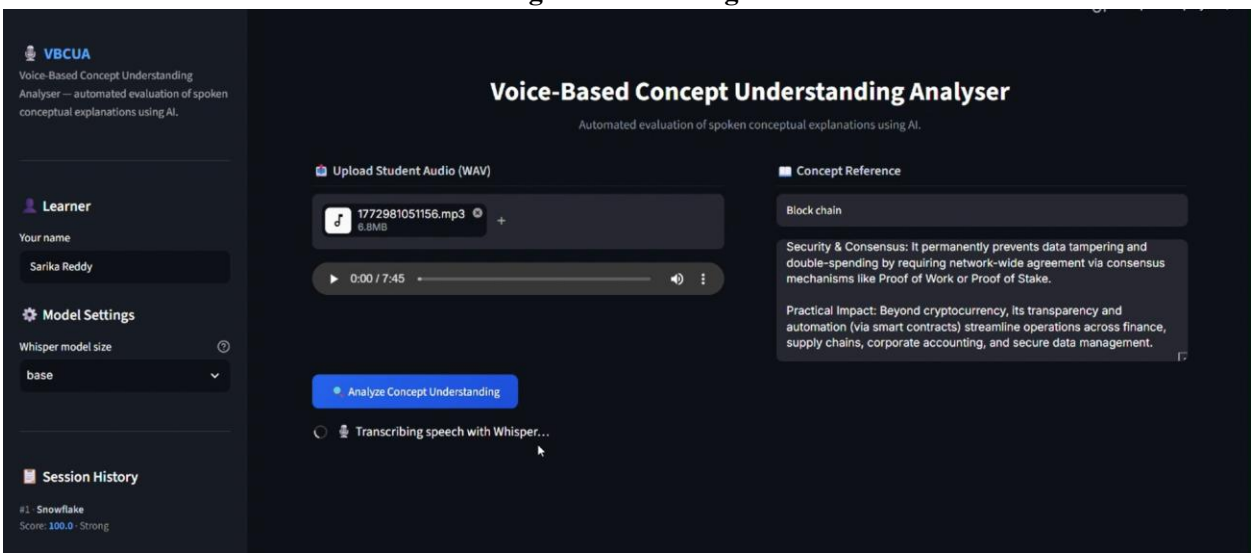


Fig 13.2: Upload of Voice and Reference Concept

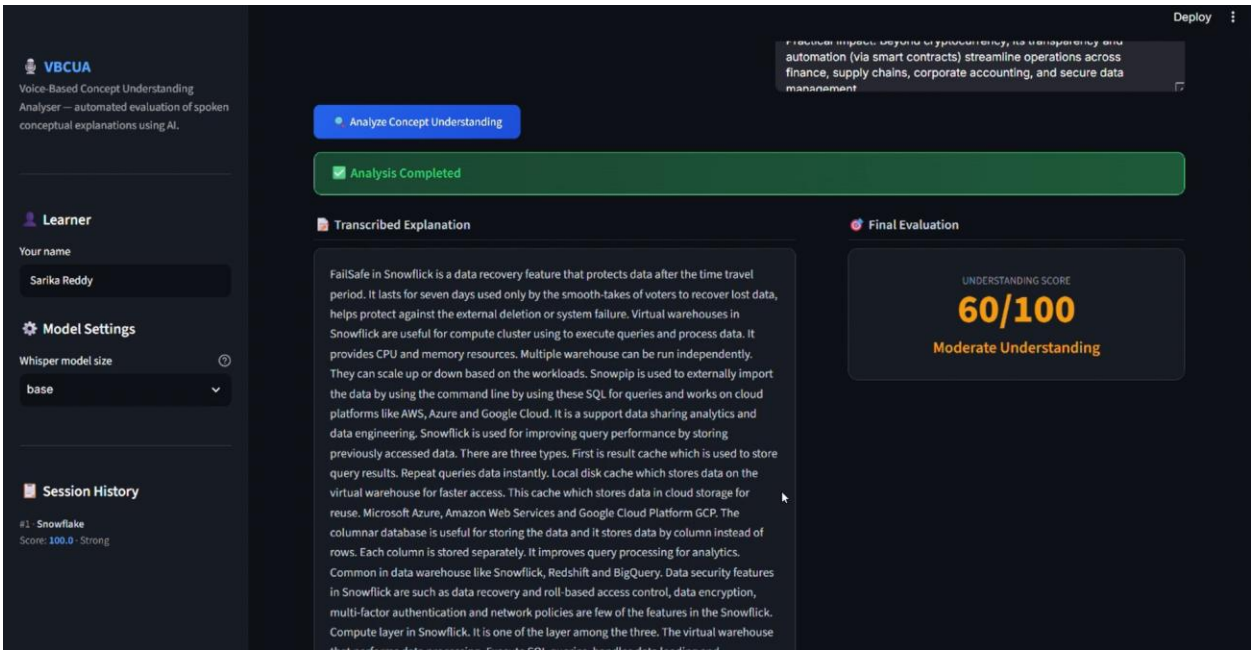


Fig 13.3: Analysis of Student Transcript Using Whisper

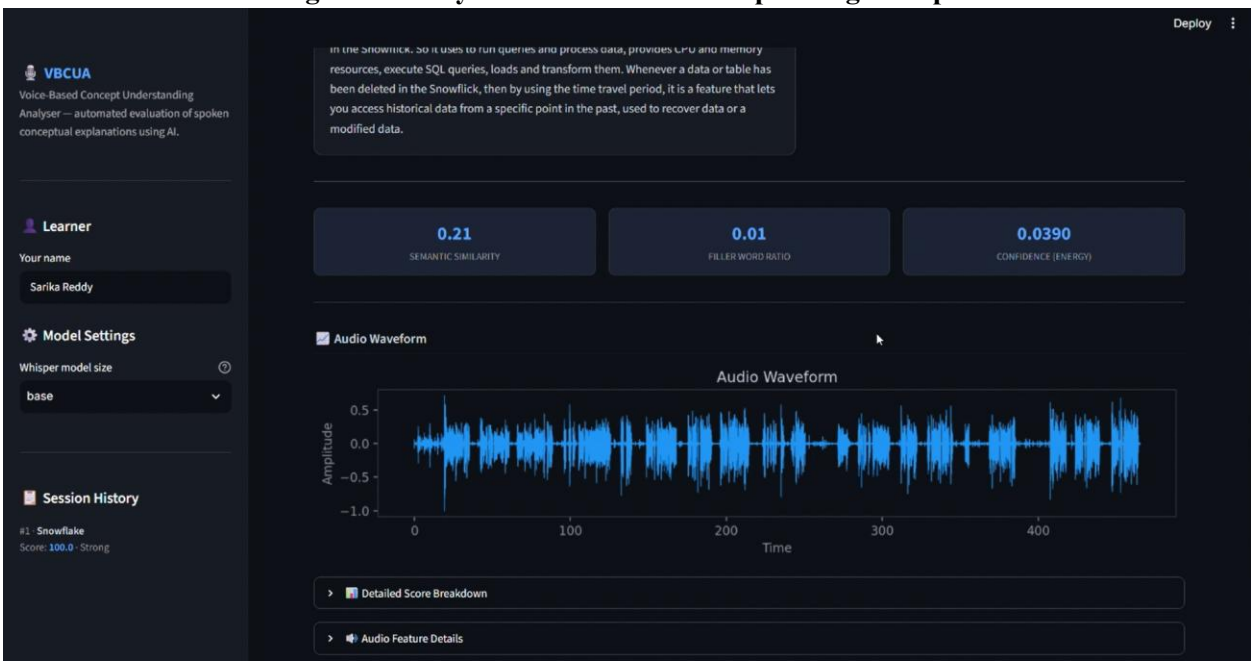


Fig 13.4: Audio wave Visualization of Voice using Mathplot

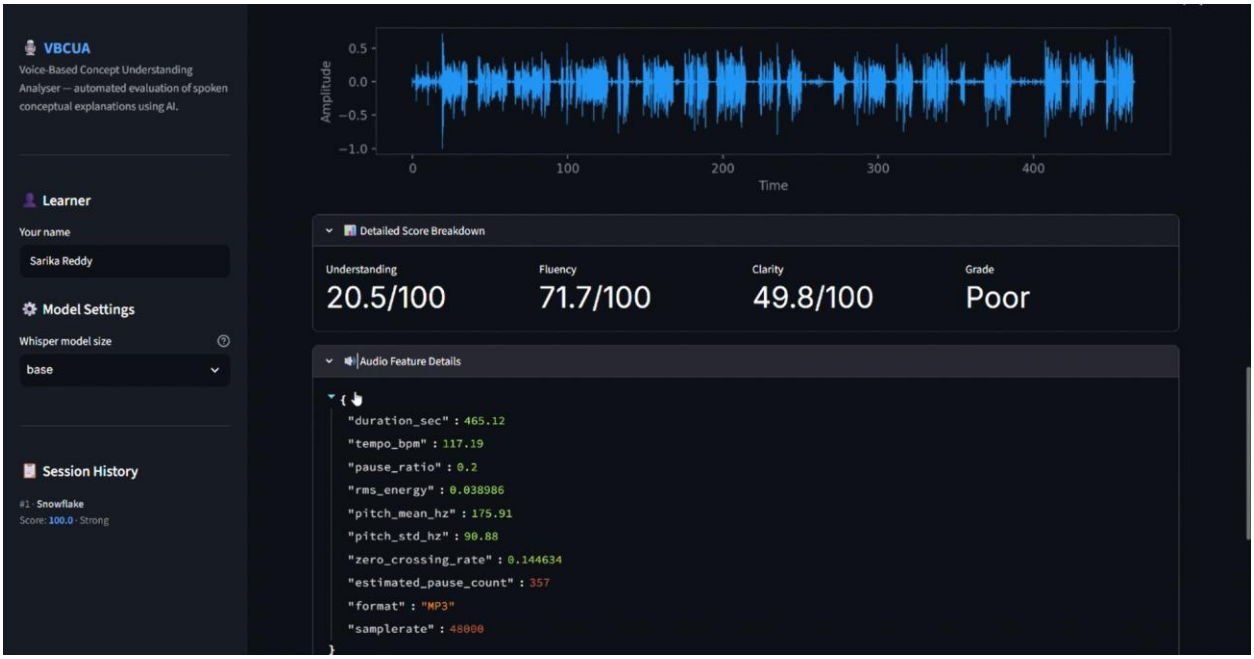


Fig 13.5: Analyzing the Details of the Students

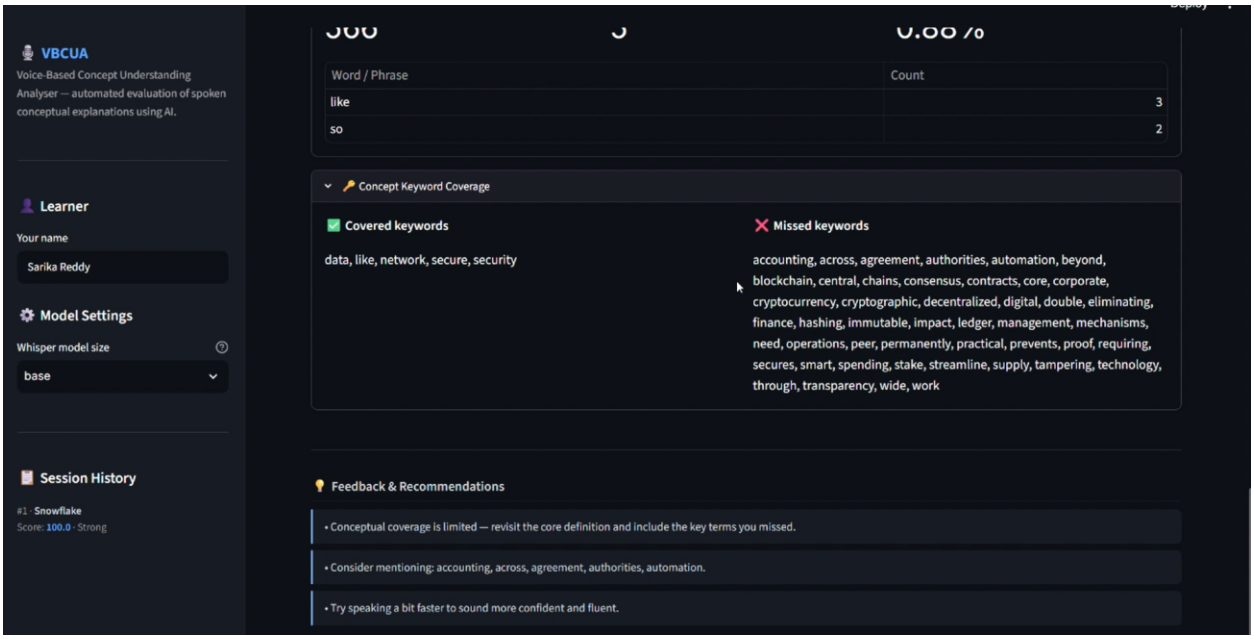


Fig 13.6: Keywords to be improved with Final Feedback

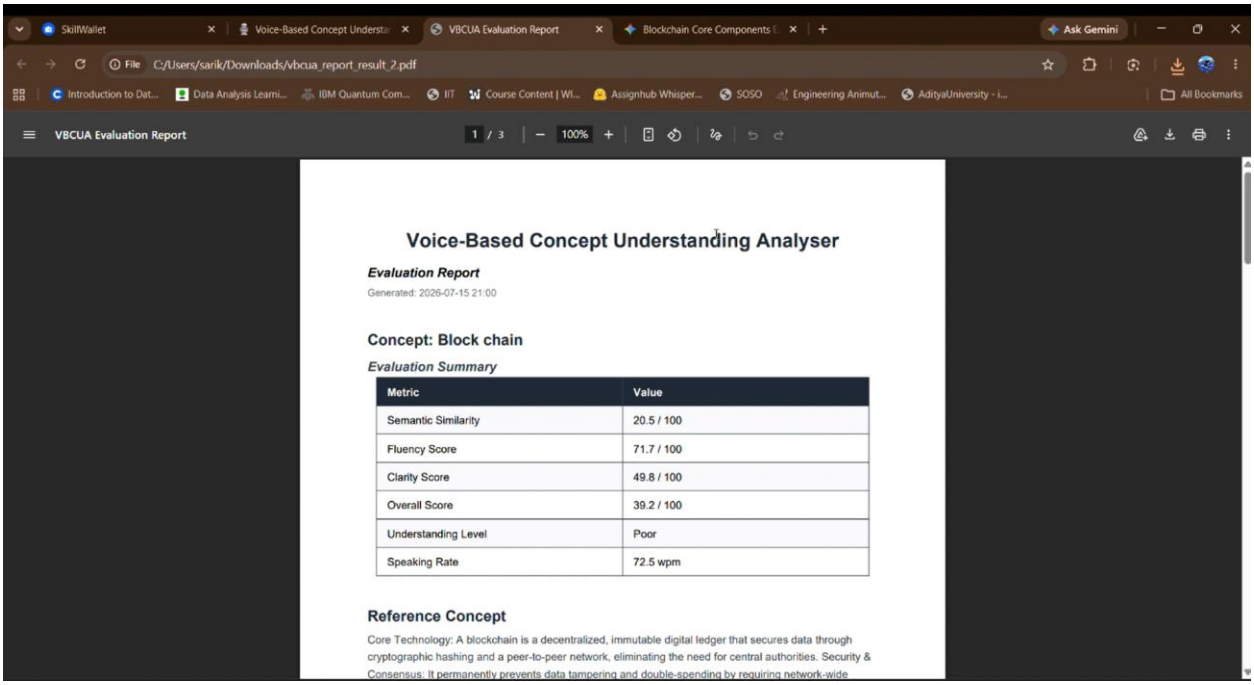


Fig 13.7: Generation of Student Statistics Report

13.1 Key Observations

- Whisper transcription accuracy exceeded 90 percent word accuracy for clearly spoken audio with minimal background noise, though accuracy dropped for recordings with heavy room echo or very low volume. The base model was selected by default for a good trade-off between speed and accuracy.
- Semantic similarity scores correlated strongly with actual conceptual coverage. Explanations covering the main definition, components, and examples of a concept consistently scored above 0.7, while vague or off-topic explanations scored below 0.4, correctly triggering the Poor Understanding classification.
- Filler word detection worked reliably for the defined word list, though a few subject-specific terms were initially flagged due to partial string matches, requiring minor regex refinement.
- Audio feature extraction via Librosa was computationally fast, typically under two seconds per file. The pause_ratio metric proved particularly useful, since high pause ratios above 30 percent correlated with lower semantic coverage scores.
- PDF report generation was reliable, with file sizes ranging from approximately 40 KB to 120 KB depending on transcript and feedback length.

13.2 Sample Score Interpretation

Scenario	Similarity	Filler Ratio	Pause Ratio	RMS Energy	Score	Level
Clear, accurate explanation	0.82	0.02	0.18	0.035	95	Strong
Moderate coverage, some fillers	0.55	0.08	0.22	0.028	65	Moderate

Scenario	Similarity	Filler Ratio	Pause Ratio	RMS Energy	Score	Level
Vague, many pauses	0.32	0.14	0.38	0.008	30	Poor
Accurate but very hesitant	0.75	0.03	0.42	0.022	65	Moderate

Semantic similarity is the most heavily weighted component, contributing up to 50 points, which reflects the project's primary goal of evaluating conceptual understanding rather than delivery alone.

14. Outcome

- Integrated Whisper (speech-to-text) and Sentence-BERT (semantic analysis) into a single processing pipeline.
- Developed AI-driven pipelines for speech analysis, semantic scoring, and fluency evaluation.
- Implemented the evaluate_understanding formula, classifying explanations as Strong, Moderate, or Poor.
- Generated automated PDF reports containing transcripts, scores, keyword coverage, and personalised feedback.
- Built a responsive, dark-themed Streamlit application matching the full project specification.
- Persisted all data in a normalised SQLite relational schema matching the entity-relationship diagram.

15. Future Enhancements

- Live microphone recording using streamlit-webrtc, removing the need to upload a file.
- Multi-language support, extending Whisper's existing multilingual capability and using multilingual SBERT models such as paraphrase-multilingual-MiniLM-L12-v2.
- Rubric-based scoring, allowing a structured rubric with weighted criteria instead of a single reference text.
- User accounts and progress tracking, including historical performance graphs.
- A teacher dashboard for uploading reference concepts, reviewing student evaluations, and exporting class-wide summaries.
- Cloud deployment to Streamlit Community Cloud or as a Docker container.
- Improved pause detection using a Voice Activity Detection approach, such as Silero-VAD, in place of RMS thresholding.

16. Conclusion

VBCUA is a working, end-to-end AI system that addresses the problem of automated spoken concept evaluation. The project integrates four major AI and machine learning technologies, Whisper, Sentence-BERT, Librosa, and ReportLab, into a cohesive pipeline, alongside a responsive Streamlit web application, a normalised SQLite database, and a comprehensive automated test suite.

Beyond the technical implementation, the project involved practical engineering challenges including dependency management, model loading performance, audio format normalisation, and the gap between

theoretical model accuracy and real-world transcription quality. The resulting codebase, with its modular design and test coverage, provides a solid foundation for the future enhancements identified above.

17. References

1. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. OpenAI. github.com/openai/whisper

2. Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019. sbert.net

3. McFee, B., Raffel, C., Liang, D., et al. (2015). librosa: Audio and Music Signal Analysis in Python. Proceedings of the 14th Python in Science Conference. librosa.org

4. Bortfeld, H., Leon, S. D., Bloom, J. E., Schober, M. F., and Brennan, S. E. (2001). Disfluency rates in conversation: Effects of age, relationship, topic, role, and gender. Language and Speech, 44(2), 123-147.

5. Foltz, P. W., Laham, D., and Landauer, T. K. (1999). Automated Essay Scoring: Applications to Educational Technology. Proceedings of EdMedia 1999.

6. Streamlit Inc. (2024). Streamlit Documentation. docs.streamlit.io

7. ReportLab Inc. (2024). ReportLab User Guide. reportlab.com

8. PyTorch Team (2024). PyTorch Documentation. pytorch.org/docs/stable

9. Python Software Foundation (2024). Python 3 Documentation. python.org/downloads

10. FastAPI Documentation. fastapi.tiangolo.com